Chapter Four

Functional Dependencies and Normalization

Objectives

- **By the end of this chapter, you should be able to:**
  - Define Data redundancy and its effects
  - Define and explain the concept of functional dependency (FD) in a relational database.
  - Identify and explain types of functional dependencies.
  - Explain the purpose and process of database normalization.
  - Describe normal forms: 1NF, 2NF, 3NF, BCNF, 4NF, and 5NF.
  - Analyze relation schemas to detect and eliminate redundancy and anomalies.
  - Apply normalization techniques to real-world examples to achieve higher normal forms.
  - Understand the trade-offs between normalization and performance.

Data Redundancy and Anomalies

- **Data redundancy** refers to storing the same piece of information in more than one place.
- It often happens because attributes from multiple entities are mixed within the same table.
- Although some redundancy may be unavoidable, **uncontrolled redundancy leads to serious issues**, including:
  - Wasted storage
  - Inconsistencies
  - Poor data integrity
- A **data anomaly** occurs in a poorly designed database due to data being stored in the wrong place or duplicated unnecessarily.

Example

- **Employee**

EmpID	EmpName	DeptName	DeptHead
101	Hana	Accounting	Alice
102	Samuel	Accounting	Alice
103	Meron	Accounting	Alice
104	Daniel	Accounting	Alice
105	Ruth	Accounting	Alice

- **Student**

StudentID	StudentName	Major	Advisor
S001	Yonatan	CS	Dr. Daniel
S002	Sara	CS	Dr. Daniel
S003	Melat	IT	Dr. Daniel
S004	Nahom	SE	Dr. Daniel
S005	Abel	CS	Dr. Daniel

Types of Anomalies

- When a table is poorly designed, operations cause anomalies.

1. **Insertion Anomaly**

- An **insertion anomaly** occurs when you cannot insert a piece of information **unless additional, unrelated information is also provided.**
- Insertion anomalies always signal that attributes from different entities are being mixed into one table.
- **Example:**

EmpID	EmpName	DeptName	DeptHead
101	Hana	Accounting	Alice
102	Sami	Accounting	Alice
103	Meron	Accounting	Alice
104	Daniel	Accounting	Alice

- If you want to add a department but there is no employee yet, you cannot insert DeptName, DeptHead without an employee.

Cont.

2. Modification/Update Anomaly

- Updating a repeated value requires changing it everywhere; otherwise, inconsistencies may occur.
- **Example:** In the table on the previous slide
  - If the department head changes from Alice to Alex , we have to update all 4 record. Missing one causes inconsistency.
  - Consider this relation:

Emp_ID	Proj_ID	Ename	Pname	No_hours
E1	P1	John	Billing	10
E2	P1	Alice	Billing	15
E3	P1	Bob	Billing	12
E5	P2	David	Accounting	9

- Changing the name of project ID P1 from "Billing" to "Customer-Accounting" may cause this update to be made for all employees working on project P1.

Cont.

3. Deletion Anomaly

- Deleting one row accidentally removes important data.
- **Example:**

EmpID	ProjID	Ename	Pname	No_hours
E1	P1	John	Billing	10
E2	P1	Alice	Billing	15
E3	P2	Bob	Accounting	12
E4	P3	Carol	HR	8
E5	P3	David	HR	9

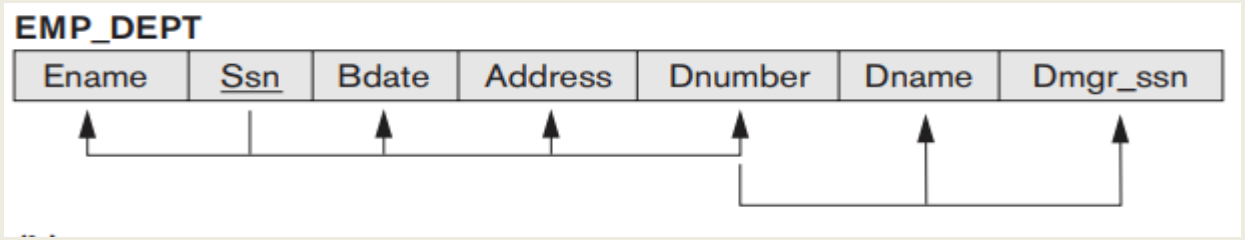
- When a project is deleted, it will result in deleting all the employees who work on that project.
- Alternately, if an employee is the only employee on a project, deleting that employee would result in deleting the corresponding project.

Functional Dependencies (FDs)

- FD is a constraint that describes the relationship between attributes in a table.
- A functional dependency X → Y means: if two rows agree on X, they must agree on Y.
- The values of Y in a tuple are determined by the values of X.
- Equivalently, X uniquely determines Y, so Y is **functionally dependent** on X.
- Example: EmpID → EmpName (EmpID uniquely determines the EmpName).
- **Terminology**:
  - **Determinant**: Left-hand side (X) of the FD
  - **Dependent**: Right-hand side (Y) of the FD
- **Importance of FDs**: Functional dependencies help:
  - Identify correct primary keys
  - Discover improper designs
  - Determine how to decompose tables
  - Measure the "goodness" of relational designs

Cont.

- **Key Points:**
  - Functional dependencies are defined by the real-world meaning of data, not by sample rows
  - Used to define normal forms for relations
  - Represent constraints based on attribute interrelationships
- Denoted graphically in schemas as the following:



- social security number determines employee name, birth date, address and Dnumber.
- Dnumber determines Dname and Department manager

Prime vs. Non-prime Attribute

- A **Prime attribute** must be a member of some candidate key
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.
- **Example**: Consider a relation with the following attributes: EmpID, Project, Hours, Manager
  - **FDs:**
    - EmpID, Project → Hours
    - Project → Manager
  - **Candidate Key** = {EmpID, Project}
  - **Prime Attributes** = EmpID, Project
  - **Non-Prime Attributes** = Hours, Manager

Inference Rules for FDs

- **Reading Assignment**
  - Read on **Inference Rules for FDs** and about **attribute closure** to gain a deeper understanding about the different rules involved in functional dependencies and normalization

Types of FDs

1. **Trivial FD:** A functional dependency is trivial when the right-hand side (dependent) is part of the left-hand side (determinant).
   - Formally: **$Y \subseteq X$**
   - **Example**: {EmpID, DeptID} → EmpID
   - Note: Trivial FDs **always hold**, no matter what data is present.
2. **Non-Trivial FD:** A dependency is non-trivial when the right-hand side is not part of the left-hand side.
   - Formally: **$Y \not\subseteq X$**
   - **Example**:
     - (A, B) → (B, C)
     - StudentID → Email
     - (StudentID, CourseID) → (CourseID, Grade)

Cont.

3. **Completely Non-Trivial FD**

- A dependency is **completely non-trivial** when:
  - It is **non-trivial** ($Y \not\subset X$), and
  - The determinant and dependent **share no attributes at all**.
- Formally: $\mathbf{X \cap Y = \emptyset}$
- **Examples:**
  - CourseID → CourseName
  - **(A, B) → (C, D)**

Cont.

4. **Full Functional Dependency**

- An FD **X** → **Y** is full if **every attribute** in X is necessary to determine Y. Removing **any** attribute from X breaks the dependency.
- It is mainly related to composite keys
- **Example:**
  - (CourseID, StudentID) → Grade
  - Grade depends on the specific **student in a specific course**.
  - If you remove either CourseID or StudentID, the grade is no longer uniquely known.
  - Removing either attribute makes the dependency invalid.

Cont.

5. **Partial Functional Dependency**

- A dependency **X** → **Y** is partial if:
  - Y depends only on **some part** of a composite key.
- It happens when we have a composite key but **non-prime** attributes depend on only part of the composite key.
- **Example:** Assume we have a composite key: (StudentID, CourseID)
  - **StudentID** → **Major** ➔This is a **partial** dependency because:
    - The full key is (StudentID, CourseID)
    - But Major depends only on part of it (StudentID)
- Note: Partial FDs cause 2NF violations and redundancy.

Cont.

6. **Transitive Functional Dependency**

- A dependency X → Z is transitive if:
  - X determines Y
  - Y determines Z
  - and Y is NOT a key.
- i.e.: X➔Y➔Z
- **Example**
  - EmpID → DeptID, DeptID → DeptName
  - Therefore: **EmpID** → **DeptName** (which is transitive)
- Note: Transitive FDs lead to 3NF violations and cause redundancy.

Cont.

7. **Multivalued Dependency (MVD):** A →→ B means:
   - For one value of A, there can be multiple values of B and B is independent of all other non-key attributes.
- **Example 1:** Consider a relation Person with the attributes: Person, Language, Skill
  - A person may speak multiple languages
  - A person may have multiple skills
  - Languages and Skills are independent(unrelated) lists → **MVD**
- **Example 2:** Product(ProductID, Color, Size) → MVD
  - A product can come in: Multiple colors, Multiple sizes
  - Colors and sizes are independent lists
- Note: Non-trivial MVDs cause exponential redundancy and lead to 4NF violations

Cont.

8. **Join Dependency(JD)**

- A Join Dependency occurs when a relation can be exactly reconstructed by joining multiple projections (sub-tables) without:
  - Losing rows
  - Adding incorrect rows
- For a JD to hold: The original table must be exactly equal to the join of its decomposed parts.
- Join dependencies typically occur when a table contains three or more attributes
- Note: Incorrect decomposition creates spurious tuples—fake rows that never existed leading to violation of 5NF

Normalization

- **Normalization** is the process of decomposing bad or unsatisfactory relations by breaking their attributes into smaller, well-structured relations.
- **Normal form** defines conditions, based on **keys** and **functional dependencies (FDs)**, that a relation must satisfy to be in a particular normal form.
- **Purpose of Normalization is to:**
  - Eliminate unnecessary redundancy
  - Avoid anomalies (update, insert, delete)
  - Ensure a logical and consistent data structure
  - Simplify data maintenance

Cont.

- **Practical Considerations:**
  - Normalization is aimed at high-quality database design.
  - Constraints can sometimes be difficult to understand or enforce.
  - Designers do **not always normalize to the highest possible form**; typically, normalization is done up to **3NF, BCNF, or 4NF**.
- **Denormalization:** The process of combining higher-normal-form relations into a base relation (lower normal form) to improve performance.

When to Normalize vs. Denormalize?

- **Normalize when:**
  - Data integrity is critical
  - High frequency of updates
  - Risk of data anomalies is significant
- **Denormalize when:**
  - System is read-heavy or primarily for reporting
  - Performance is more important than strict consistency

First Normal Form (1NF)

- A table is in **1NF** if:
  - Each column contains **atomic** values ➔ No multi-valued attributes
  - Columns contain values of the same type
- **Example:** Table Not in 1NF ➔

Student

StudentID	**Name**	**Phone_no**
101	Sofia	0910…, 0920…., 0975…
102	David	0911…, 0960….

- When a table is not in 1NF:
  - Data becomes difficult to query and to update.
  - **Example**: How do you get Sofia's one phone number or update only her second phone number?

Cont.

Correct 1NF Table ➔

StudentID	Name
101	Sofia
102	David

StudentID	Phone_no
101	0910…
101	0920…
101	0975…
102	0911…
102	0960…

Or alternatively

StudentID	Name	Phone_no
101	Sofia	0910…
101	Sofia	0920…
101	Sofia	0975…
102	David	0911…
102	David	0960…

Second Normal Form (2NF)

- A table is in **2NF** if:
  - It is already in **1NF**, and
  - **No partial dependency** exists (instead, it **must be full FD**)
- **Example:** Table Not in 2NF

Enrollment

StudentID	**CourseID**	**StudentName**	**Major**	**Grade**
1	DB	Sara	CS	A
1	AI	Sara	CS	A-
2	DB	Bereket	SE	B

- **Partial dependencies:**
  - StudentID → StudentName
  - StudentID → Major
- Both depend on only part of the composite key.

Cont.

Correct 2NF Decomposition

- Create new table for attributes that are partially dependent

Student

StudentID	StudentName	Major
1	Sara	CS
2	Bereket	SE

Enrollment

StudentID	CourseID	Grade
1	DB	A
1	AI	A-
2	DB	B

Third Normal Form (3NF)

- A table is in **3NF** if:
  - It is in **2NF**, and
  - **No transitive dependencies** exist
- 3NF removes dependency between non-key attributes.
- **Example:** Table Not in 3NF

StudentID	**DeptID**	**DeptName**
1	CS	Computing
2	SE	Software
3	SE	Software

- **Transitive FD**: StudentID → DeptID, DeptID → DeptName

Cont.

Correct 3NF Decomposition

- Keep only attributes that depend **directly on the key**

Student

StudentID	**DeptID**
1	CS
2	SE
3	SE

Department

DeptID	**DeptName**
CS	Computing
SE	Software

Boyce–Codd Normal Form (BCNF)

- A stronger definition of 3NF—called **Boyce-Codd normal form (BCNF)**—was proposed later by Boyce and Codd.
- A table is in **BCNF** if:
  - It is already in **Third Normal Form (3NF)**
  - **For every functional dependency X → Y, X must be a super key.**
- In simple terms: The **left side** of **every** non-trivial **FD** must be a **super key.**
- **When is BCNF needed?**
  - When 3NF still allows anomalies.
  - When there are overlapping candidate keys.
  - A non-key attribute determines part of a key
  - An attribute that is not a key acts as a determinant

Cont.

- **Why BCNF Is Stricter Than 3NF**
  - 3NF allows an FD if:
    - X is a key **OR**
    - Y is a prime attribute
  - BCNF only allows:
    - X **must be a key**.
- **Example 1:** Table Not in BCNF

CourseAssignment

StudentID	**Course**	**Instructor**
S01	DB	Ali
S02	DB	Ali
S03	DB	Sara
S01	C++	Dave

- **Candidate Keys**
  - **(StudentID, Course) ➔ PK**
  - (StudentID, Instructor) **➔what happens if this is selected as PK?**
- **Functional Dependencies**
  - (StudentID, Course) → Instructor
  - Instructor → Course
- **Why it is in 3NF?**
  - **Course is a prime attribute**
- **Why it is NOT in BCNF?**
  - **Instructor** is **not a key,** yet it determines Course
- **Note**: Assuming an instructor only gives one course but can teach different students.

Cont.

- **Example 2:** Table Not in BCNF

Teach

Course	Room	Lecturer
DB	R1	Ali
DB	R2	Ali
OS	R2	Sara
DB	R3	Meklit

- **Functional Dependencies**
  - Course, Room → Lecturer
  - Lecturer → Course
- **Candidate Keys**
  - **(Course, Room)➔PK**
  - (Room, Lecturer) **➔what happens if this is selected as PK?**
- **Why it is in 3NF?**
  - In Lecturer➔Course, Course is prime
- **Why it is NOT in BCNF?**
  - Lecturer is **not a key** but determines Course
- **Note**: Assuming the same room **can't** be used by the same course more than once, but can be used by different courses. And assuming a Lecturer only gives one course but can teach in different rooms.

Cont.

BCNF Decomposition

- Whenever **X → Y violates BCNF**, **split into the following tables**
  - X ∪ Y
  - (Original table − Y) ∪ X
- **Example 1:**

InstructorCourse

Instructor	Course

StudentInstructor

StudentID	Instructor

- **Example 2:**

LecturerRoom

Lecturer	Room

LecturerCourse

Lecturer	Course

Fourth Normal Form (4NF)

- A table is in **4NF** if:
  - It is in **BCNF**, and
  - It has **no nontrivial multivalued dependencies** (MVDs)
- **Example:** Table Not in 4NF

Person

Person	Language	Skill
Eric	English	Cooking
Eric	English	Painting
Eric	French	Cooking
Eric	French	Painting

- Language and Skill lists are **independent**
  - Person →→ Language
  - Person →→ Skill
- Simply stated: A table should not contain two or more independent multi-valued facts about the same entity.

Cont.

Correct 4NF Decomposition

- Decompose into two tables: R1(A, B)& R2(A, C)

Person–Language

Person	Language
Eric	English
Eric	French

Person–Skill

Person	Skill
Eric	Cooking
Eric	Painting

Fifth Normal Form (5NF)

- Also called Project–Join Normal Form – PJNF
- A table is in **5NF** if:
  - It is in **4NF**, and
  - Every join dependency in the table is implied by the candidate keys
- All decompositions based on join dependencies must be **lossless**
- **Example 1:** Table Not in 5NF

Supplier	Part	Project
S1	P1	pJ1
S1	P2	pJ1
S1	P1	pJ2

- Wrong decomposition causes **fake tuples**.

Cont.

Correct 5NF Decomposition

- Decompose a relation into n relations based on a non-trivial join dependency such that the decomposition is lossless.
- **Example 1:**
- **Supplier_Part➔**

Supplier	Part
S1	P1
S1	P2

- **Supplier_Project➔**

Supplier	Project
S1	pJ1
S1	pJ2

- **Part_Project➔**

Part	Project
P1	pJ1
P2	pJ1
P1	pJ2

Cont.

- **Example 2:** Table Not in 5NF➔

Agent	Company	Product
ABC	Samsung	Smartphone
ABC	Samsung	Smart TV
ABC	LG	Smart TV

Correct 5NF Decomposition

- **Agent_Company ➔**

Agent	Company
ABC	Samsung
ABC	LG

- **Company_Product➔**

Company	Product
Samsung	Smart TV
Samsung	Smartphone
LG	Smart TV

- **Agent_Product**

Agent	Product
ABC	Smartphone
ABC	Smart TV

Sixth Normal Form (6NF)

- **Reading Assignment**: Read and understand about Sixth Normal Form and when it might be needed.

Final Considerations

- Ensuring each relation is in a specific NF alone **does not guarantee correctness** of the overall database.
- We must also consider properties of the **decomposed schemas as a whole**.
- **Essential properties after decomposition:**
  - **Lossless (Nonadditive) Join:**
    - Ensures that **no spurious tuples** are generated when relations are joined.
    - This **must** always be satisfied.
  - **Dependency Preservation:**
    - Ensures all **functional dependencies** are represented in the decomposed relations.
    - **Sometimes difficult to achieve**, so it is some times sacrificed